Introduction

Hello, everyone. And welcome to lecture 7 of CME 295. So today, we're going to focus on practical techniques to let our LLM interact with the outside world with other systems. Because up until now, our LLM was purely on its own. We've trained it. We've seen how it can reason on problem math, coding math, coding problems. And now, what we want to do is to use our LLM in the context of other systems. So today's class, we'll focus on RAG, that you may have heard, tool calling, and agents. But before we start, as usual, I'm going to recap what we did last time.

So if you remember, last time, we focused on reasoning models. And we saw the differences between reasoning model and what we call the vanilla LLM. And in particular, up until the lecture before last lecture, what we saw was we fed a prompt to the LLM, and it gave us directly a response. But what we saw last time was that if we let the LLM reason before outputting the response, then we can gain some performance when it comes to reasoning tasks, such as math and coding. And so, in particular, reasoning models, what they do is they take a prompt as input, and then what they output is both a reasoning chain, which is typically hidden from the user, and then a response.

So with that, we saw how we could train a model to be more of a reasoning model. And in particular, we saw a core RL algorithm called GRPO, which stands for Group Relative Policy Optimization. And we saw that this algorithm had some differences compared to the ones that we saw previously. And in particular, one notable aspect is that it does not have-- it does not train a value function. So here, this illustration shows a little bit how GRPO is trained. So it takes a query as input. And then it computes an advantage for each output by computing the rewards for different completions of a same prompt. And then computing a quantity, which is the advantage, that is relative to the other rewards of that group of completions.

And then we saw that if we applied GRPO with carefully chosen rewards, which is one, rewarding the model for outputting a reasoning chain, and then second, rewarding the model for producing a good response. What we saw is that as the RL training progresses, we have an improvement of the model on these reasoning tasks. And so we saw one of the tasks being math problems. So here on the left graph, you see the evolution of the performance of the model on the aim data set, which is a challenging math problem. And we saw that, but we also saw that the model kept on outputting responses that were longer and longer. And in particular, we saw that even though

towards the end of the graph above, that the performance was plateauing. We saw that the output length was still increasing.

So then what we did was go back to the loss formulation that is used by GRPO and realize that there is a term that makes the contribution of a token different, if it is in a short response or a long response. So this is a phenomenon called length bias. And we saw some mitigation strategies that were explored by some papers that came out in the past few months. So one was DAPO, which had a normalization factor that was not dependent on where the token was located and which sentence it was located. And the other one was this paper called "GRPO Done Right," which actually just removed the normalization term. All good on that? Cool.

So this was last time. And last time, what we said right before starting the reasoning class was to enumerate the strengths and the weaknesses of vanilla LLMs. So last lecture was all about focusing on how we can improve the limited reasoning capabilities of vanilla LLMs. And in this lecture, what we will do is two things. So the first one is see how we can connect our LLM to the ever evolving knowledge base, and, in particular, see how we can have access to the latest information. And then the second one is how our LLM can help us perform actions. And we will see this with Shervine with things like tool calling and agentic workflows.

Cool. So with that, let's start with the first one. And let's start with this method called RAG.

RAG Overview

RAG overview that you may have heard. So let's suppose you have a model that you have trained. But the problem is that the pre-training data on which you have trained your model is, let's say, a month ago, let's suppose. Now, let's suppose you want to prompt your model about the winner of the elections that happened a couple of weeks ago. Well, your model will not be able to respond to you, or it will output the incorrect answer because up until now, our LLM does not have any link to outside sources. It only relies on the knowledge that it has acquired during training. So the response that it will give us will only be based on the data that has been trained up until the cutoff, which is a month ago in this example.

And so we have this big limitation, which is our LLM only knows about things that it has been trained on. And you will see that all the models out there. So here, I have an example with OpenAI GPT-5. So if you look at their model cards, they always have

these knowledge cutoff dates that is written somewhere. And in the case, for instance of GPT-5, the knowledge cutoff date is September 30, 2024, which means that if you ask it in a very naive way, anything that happened after that, the base model will not be able to answer you as is.

Well, you may tell me why not just continue training your model on data that happened after that? Well, the problem with that-- actually, there are several problems with this. So the first problem is that it's very tricky to change the knowledge of an LLM without causing regression on other things. So this is typically a task that people, they try to avoid doing. And the second thing is it's not very practical, because you may very well have use cases that require you to fine tune this model. So let's suppose you have a use case one, you fine tune from this model. And then somehow you want to update the weights of your model to inject some knowledge. Well, you somehow will have to do that for all the use cases that you are doing, which basically adds a lot of overhead for you and just adds a lot of maintenance. So people, they typically prefer to not do additional training to inject knowledge.

So one idea can be to somehow take your prompt and just add anything that happens after the cutoff date as a way for your model to just know what happened. Well, the problem with that naive approach is that as you know context length is limited. And so typically, models have on the order of magnitude of hundreds of thousands of tokens in context length. Do you know what that is roughly-- what it is roughly equal to? Yes. So one token is equal to four characters. So using this rough approximation, hundreds of thousands of tokens is roughly like hundreds of pages, something like a very big book. It's big, but it's not enough for us to go in that very naive route.

So again, going back to GPT-5, so if you go to the model card, you have the knowledge cutoff date, which is September 2024. You also have the context window. And in this case, it's 400,000 tokens. So let's suppose actually, context is not a problem. It's actually unlimited. Let's imagine we actually put everything in the context. Well, the problem then is that people noticed that if you feed a lot of irrelevant information to your LLM, the performance of the LLM will actually degrade. Meaning, that if for instance, you ask it about, I guess, who was the winner of the last elections? And then you feed it a bunch of information that are not relevant, your LLM will tend to be confused.

And so people have run these tests. That's called the needle in a haystack test, where the idea is you give a big prompt to your LLM, which is your haystack, and you place a fact in the prompt. And you ask your model what that fact was. So the idea is for your

LLM to know, I guess, among that huge prompt, where is the relevant information, which is the needle. And so when people have tried doing this for several length of prompts and tried different positions of where to put the facts, they have seen that the length of the prompt and the position at which you put the facts are both important.

So here on the slide, we have a heat map that was performed for GPT-4, which was I guess one or two years ago. And what the person did was placed the fact at different places in the document. So this is document depth, and the x-axis is the length of your prompt. And what we saw was that for prompts that exceeded a certain amount of tokens, the LLM actually had trouble retrieving the correct piece of information. And in particular, it had trouble doing so when the fact was somewhere in the first half of the prompt. So this just tells us that, even if, let's say, our context length was unlimited, we would still have a problem by just going through that naive approach. So that's another reason.

So now, let's suppose, context length is unlimited. Let's suppose the problem that I mentioned is not a problem. Well, the other problem is that you pay. So in particular, these calls, these LLM calls, they are per token. So the bigger your input prompt, the more you will pay. So you have an incentive to not put too much in your prompt just from that standpoint. And so for instance, again, going back to GPT-5, order of magnitude is somewhere around $1 per million token. So I guess it's not that expensive, but it can add up, if you do that for all your prompts.

So for all these reasons, I hope I convinced you that we need a more clever approach, where instead of putting all the new information all at once in the prompt, what we do is we only somehow find the relevant information and put that in the prompt. So that is the idea behind RAG. RAG stands for Retrieval Augmented Generation. And the idea here is to augment the prompt with relevant information. And here, I put relevant in bold. And this is the, I guess, the core part of this technique is how can we get only the relevant part in the prompt? So we'll see that in a second.

So just at a very high level, so you have, let's say, a question as input. So in this case, who was the winner of, let's say, the local election? The idea here is to somehow fetch the correct or the relevant piece of information and then augment that here in order to output your answer. So that's the rough idea. So does this method make sense so far? Yeah. OK. Cool.

So that is the idea behind RAG. And now, we're going to go into more details. So what I mentioned is the rough idea. And here, I just want to emphasize on the three main

steps of RAG. So first one is you have your prompt, and you somehow want to retrieve a relevant piece of information that will help you in answering your prompt. So here, the first step is to retrieve relevant documents. And so you can think of your prompt as being one entity. And then you can have some other space, which maybe, I don't know, knowledge base, where all your documents live. And so the idea is to somehow fetch the relevant documents. So this is the retrieve step.

The second step is once you have fetched the relevant information, you augment your prompt. So you take that retrieved info. You just put it in your prompt, and then ask the question. So in the local election example, it's as if I was saying, who is the winner of this election? And then I retrieve the relevant piece of information. And now, the prompt becomes, who is the winner of this election? And by the way, this election was held, blah, blah, blah. And this was the winner. And this is what we're feeding to our LLM. So in other words, we're giving the answer in the prompt.

And the third step is to feed that prompt to the LLM to generate the response. Yeah? Exactly. So the question is, you may very well somehow do a bad job at retrieval stage, so yes. So this is why the retrieval stage is so important. And we're going to focus on what we can do to make sure that one part does well. We'll see how we can evaluate, I guess, our setup and different methods. But when we talk about RAG, we're mainly focusing on making the retrieval parts as good as it can. Cool.

And I just want to emphasize once again on why it's called RAG. So you have retrieve, augment, generate-- RAG. Cool. And as you pointed out, the first step, which is the retrieval step is very important, which is why we'll spend a little bit of time over there.

So I guess the first step is for us to somehow clean the set of documents that we may need. So I said, we may want to look into outside information, but we need to somehow sort that order that put that somewhere. And this whole thing is usually called a knowledge base. So in order to form our knowledge base, what we do is typically collect the set of documents that are or may be useful. And once we do that, what we do is we divide them into, what we call, chunks. So a chunk is you can think of it as a subset of the document which has a given maximum length, which is are measured in number of tokens, which is typically on the order of hundreds of tokens. And the idea here is whenever you hear retrieval, you should think about embeddings. And here, what we do is we compute embeddings corresponding to each of these chunks.

Now, when you create your knowledge base, there are a few hyperparameters that you need to tweak. So the first one, obviously, is the size of the embedding. So typically, you would want a bigger size, if, let's say, your documents are maybe more nuanced, more complex. But then if you have a higher size, maybe it will take more space. Maybe you'll have more computation at inference time. So I guess it's a trade-off. You don't necessarily want too big of an embedding size. So here, typically, embedding sizes are on the order of thousands. So for instance, like 1,500 something like this.

So then you have the chunk size. Chunk size is how big your little pieces here are. So you don't want them to be too small, because, otherwise, the text may be out of context. You don't want it to be too large, because maybe the embedding will not represent, in a meaningful way, what is inside. So again, it's a trade-off. But typically, people they choose chunk size of around 500 tokens, like, on the order of hundreds of tokens.

And then you also have a-- oh yeah. You have a question. Yeah? So the question is, do you train an embedding model for this. So you have two choices, either you can use a pre-trained embedding model, which people typically do, or you can train your own. We will see that in a bit more detail in a few slides. So the question is, what is the purpose of the embedding model? So we will see this in a second. But long story short, it tries to represent chunks, such that it achieves your end goal, which is to fetch relevant documents. So we will see a little bit how they're trained. But this is the general idea. Cool.

So that's this. And then we have a third hyperparameter, which is how much overlap you want to have in between your chunks. So here, when you do the division, in a very naive way, you have everything be independent, no overlap in between. But typically, you have some part that is from the previous chunk that is relevant to understand the current chunk, which is why we want to have some overlap, which is why people, they typically also have that. So it's typically in the low hundreds of tokens. Cool.

So let's suppose you have your knowledge base. Now, the question is, given a prompt, how can you retrieve relevant documents? And the answer to that is we typically proceed in two steps. So I'm not sure if any of you has a background in recommendation systems or search. Does any? Yeah. So the methods we're seeing here are very similar to that space. So I guess people in the LLM community. They have borrowed ideas and just leveraged some techniques that we have over there. And this is typically, a setting that will also have for recommendation problems.

So we have two stages. So the first stage is typically called candidate retrieval. And the goal here is to go from a set of many, many, many chunks, and filter it down to a much smaller set of potentially relevant candidates. So during that stage, what we're trying to do is to somehow maximize recall, just do a rough operation, so that we get as many potentially relevant candidates as possible.

And then we have a second stage, which is sometimes optional. But this stage is to really make sure we have the top documents being really the relevant ones. And this one is called ranking. So the idea here is based on the list of potentially relevant documents, to really rank them in a way that really the relevant ones come at the top and so on. And typically, during that stage, we're going to use a model, a method that's going to be a bit more compute intensive because we have a much smaller set of candidates to rank compared to the first one.

So going back to your question on how do we want our embeddings to be? So here, it will really impact the first stage. And we will see that in a second. But the second stage is also quite important. Cool.

So far so good? Is everyone clear with the two-stage approach? Yeah. Very good question. So the question is, do we chunk things in a naive way, as in we just go with the number of tokens regardless of what happens? So it's a great question. And the answer is that we will see some extensions that will mitigate the problem of when you chunk it in a way that does not make sense in a naive way, you want to somehow put that into context. And we will see a method that does that. So in a few slides, we will see that. But I think your question is also a great question, because depending on the kind of document that we have, for instance, if we have, I don't know, like a JSON file or a markdown or depending on the file that you need to chunk, you also need to be aware of the structure that is within those files. So there is also some nuance there that we will not go into details, but I just want to call that out. But, yeah, great question. Any other questions? OK. Cool.

So now that we're clear on the two main stages of retrieval, we're going to focus on each one of these steps. So as I mentioned, the first step is candidate retrieval.

Similarity Search with SBERT and Bi-encoders

Similarity search with SBERT and bi-encoders. So here, what we want is among that potentially huge knowledge base to somehow filter it down to, let's say, over 100

potentially relevant candidates. So here, what we do is well, we will leverage the embeddings that I guess computed during the knowledge base initialization. And we will try to fetch potentially relevant candidates by doing a semantic similarity search.

So do you recall how we compare embeddings? Yes. So cosine similarity is typically one way one good way to compare embeddings. So the idea here is to represent our query with an embedding. We already have embeddings of all our chunks. So the idea here is to somehow find the most relevant chunks by doing this similarity search and filtering out the ones that come at the top. So the idea here is you have your query, you have your chunk. both of them, you find an embedding. And then you perform a similarity operation, which is most of the time cosine similarity. And you obtain a similarity score. So the idea here is you just keep top, I don't 100, and you go with that.

So I just want to call out that there is some complexity in that stage, because your knowledge base can potentially be huge. So what people do is typically use what we call approximate nearest neighbor methods. So you may have heard of some libraries that do that. So typically, this is something that will be relevant here. We're not going to go into details, but I just want to call that out. So here, the idea here is that when you build your knowledge base, you somehow partition the embeddings in a way that will avoid-- like make you avoid doing like just a naive linear search. So that's the idea. But you may see some techniques like ANN techniques, approximate nearest neighbor techniques. And these are typically happening here.

So another thing that I want to point out is the name of the architecture that we typically use here that you may also hear. And for that, we need to recall that these embeddings, they're actually obtained by passing them through a model. So typically encoder only. So you may hear the term BI encoder. And this one refers to the fact that we are passing the query through an encoder and then passing the chunk through an encoder. So both of them are independent. And we're comparing the embeddings. So this is another question I wanted to ask you. But I guess I didn't get the chance to. If you remember, I think lecture two or three, we had seen the BERT model. And so typically, you would have something like a BERT-like model that you would use to encode these documents.

So going back to your question, how do you compute these embeddings? So there is a paper that I highly recommend reading. Actually, that's called Sentence BERT. And so that paper explains-- so it's, first of all, it's an extension of BERT, as the name suggests. And it's an extension that allows you to compute an embedding per, let's say, sequence for your query, for your document, that is tailored to be used for

similarity search purposes. So the idea here is to have a loss function that will incentivize having a high cosine similarity for relevant entities and low cosine similarity for entities that are not relevant. So yeah, so feel free to check that paper out, if you know BERT, which I know right now, you do, it's quite easy to read. So highly recommend.

So far so good? Yeah? Yeah? So the question is, what is the default way to compute the similarity? So yes, it's cosine similarity. But again, you will see in different implementations that people can use other distances. And I would encourage you to think about how they relate to one another. So you will see, for instance, the L2 distance. But then if everything has a norm of 1, there's a lot of simplifications that can happen. So you may see some variants, but I would say they're all more or less cosine similarities. Yeah, great question. Cool.

So we're still at the candidate retrieval stage. And what we saw was one way of retrieving documents from a similarity-- sorry from a semantic similarity standpoint. So by the way, what does semantic similarity mean? It means is finding documents or finding entities that have the same meaning or that are relevant, but in the way that we compute these embeddings, we're not enforcing any kind of keyword match, like when we retrieve documents in this way, it can very well be that the documents that are matched, they do not have any word in common, but they mean the same.

Heuristic Search with BM25

Heuristic search with BM25. Well, sometimes you want to ensure that what you're looking for, what you're searching for, is exactly containing the keywords that is in your prompt. And in that case, you would want to have a second way of doing things. So you may have seen BM25 out there. So BM25 is a relevant score that is actually a heuristic score. It is based on some function of the overlap between what is in your query and what is in your document. And so that one is actually quite handy for cases, where you have a query where you absolutely want to have documents that contain keywords of this query.

So here, I have an example that I actually passed super briefly for the previous one. And we will come back to it. But let's suppose we have let's say two teddy bears. One is named Cuddly and the other one is named Huggy. So what you want is to figure out where is Cuddly. So this is your query. So if you use BM25, well, the answers that you are going to get are by definition going to contain some overlap of words that were in your query. And so here, you will have, let's say documents that contain, let's say

where cuddly is. But if let's say you only used these semantic similarity search, you would not have that guarantee. You would only have documents that are semantically similar. And those they are not guaranteed to contain keywords of your prompts. And so just to illustrate that. So here, Huggy and Cuddly, they can be thought of semantically similar. So you will probably not have cuddly-- you will not necessarily have cuddly in there. Just to illustrate that.

And that is the reason why nowadays, what people do is to look at the use cases that they have and think about whether having some heuristic, as well in the relevance score is useful for their use case. So some people, they go with the hybrid combination of this embedding-based search and the heuristic-based search. So some combination of embeddings and BM25 in which case you may have. Even more relevant documents, depending on your use case. Does that make sense? Yeah.

So now I'll come back to what you mentioned about whether cutting chunks in a naive way will necessarily lead you to things that are coherent. Well, you're completely right. Sometimes you will not. But before we answer this question, we actually are going to address another concern, which is that

HyDE and Contextual Retrieval

HyDE and contextual retrieval typically, when people want to ask about something in their LLM, the query that they input is of a different nature compared to what is in the knowledge base. So your query is typically going to be maybe something short, maybe a question. But what is in your documents is typically going to be longer. These are like sentences and sentences. So if you really think about it, if you use the same encoder to embed your query and to embed your documents, well, these two embeddings, they're not super comparable, because one is for a question and the other one is for a document.

So there's one extension that tries to mitigate that issue. And I link the paper down there. So it's called the height. So what it does is instead of computing the embedding related to the prompt, it will first generate a fake document. So it's just an LLM call, a fake document based on that prompt. And then embeds that fake document to find relevant chunks. So it may or may not work. It's not used all the time by everyone. So I would say just something that is good to try to see if that works. But this is one way of mitigating this, another way could be to simply have encoders that are specifically trained to encode the query on one side and encode the documents on the other side.

In other words, to not use the same encoder. People typically don't do that, just because of maintenance purposes. But this could also be another solution.

So now, finally going to your question regarding how we can make sense of these chunks. If they are taken out of context, they may not make sense. And so here, the idea here is to prepend some piece of text that just sums up what you need to in order to understand that chunk. So here, the idea is that you have all your documents. Let's say you have one document, that you divide it into n chunks. The idea here is instead of considering these chunks separately, you're going to compute some kind of context that is relevant to each chunk, and that is based on the whole document.

How are you going to do that?

Prompt Caching

Prompt caching. Again, the LLM call. So what you do is typically, have, let's say, the whole document. And then you have the chunk that you want to contextualize. And you ask your model, well, please give me a short succinct context to just make sense of that chunk. And now you may tell me, well, that's a lot of LLM calls. You have potentially a lot of chunks. And that's just going to be very pricey. Well, there's one strategy to make this less expensive, and I'm not sure if you've heard that option. It's called prompt caching.

So now that you very well how LLMs work, you know, typically, these are decoder only and so on. So you know that if you use the same prefix for all your prompts, well, it's going to be the same computations that you just do again and again and again. So the idea here is you just do it once. And you save all the relevant activations. And instead of computing them again, you're just going to look them up, just do a lookup, and then decode the rest. Does that make sense? Yeah.

The question is activation from a language model. Yes, because when you feed a prompt to your model and you ask it to generate a response, what the model needs to do is well, to take all of this input and then compute the activations of all the layers, and then have for the generation process to have this attention across all these other components. Well, given that it's decoder only meaning, it's only left to right. The thing that you input, if it's the same, then it will lead to the same activations. Yeah.

The question is, what if you have a closed model where prompt caching, and I'm going to just talk about this in just one slide, is an option that is closed models or providers

offer? And what they tell you is, well, this is the same prefix for all your prompts. So what we're going to do is we're just going to make it cheaper for you. So if you look at the model pricing page, you will see that there is a price for regular inputs. So inputs that are not cached. And then you have the price per cached input token. And here, you see for the let's say, OpenAI model, it's 110th of the price. So I guess, what do I want to tell you by this? Well, just try to be smart with the prompts and try to gather all the things that are likely to be repeated across prompts in the beginning. So that you can leverage this nice percentage off. Does that make sense? OK. Cool. Great.

So up until now, we have seen how we can go from potentially thousands or even, let's say, millions of chunks up to, or down to, let's say, hundreds of potentially relevant chunks. Now, what we want to do is to sort them in a more meaningful way. And the second part is more optional, because maybe sometimes this first cut that we've done may be good enough, but I guess this second step is about being more intentional in how we give the final score to be able to really select the final, let's say, top k chunks. So the second stage is called ranking or even re-ranking.

Re-ranking with Cross-encoders

Re-ranking with cross-encoders. Re-ranking because, I guess, with the first step, you already have some ranking. So we're re-ranking. And what we're doing is instead of using this very quick operation, similarity operation between embeddings that we've computed, we're going to use something that is maybe a bit more sophisticated. So instead of considering the query and the chunk separately, what we're going to do is to actually put them both in the encoder, both of them, and have a relevance score out of that. So the reason why it may be a little bit more meaningful to do it this way is that you have a model that takes a look at both your query and your chunk at the same time and gives you a score. Whereas in the first step, you had one embedding for the query and one embedding for the chunk, which didn't have that interaction that a model could capture. And you will also see out there that this setup is called cross-encoder setup because you have both your inputs fed to your encoder. So there's some cross interactions. So if you remember the first approach is a bi-encoder setup. And this one is a cross-encoder. Yeah.

The question is, you will actually compute the attention between the two. Yes, absolutely. And here, sentence there they have a lot of good documents. So I highly recommend just reading their docs. They're at the bottom of the slide. Cool. Well, you do that on all your potentially relevant chunks. So you have this score that is computed

for each of these chunks with the prompts. And then you finally obtain the ranking. And now the question is, are you happy with the ranking?

Retrieval Evaluation with NDCG, MRR

Retrieval evaluation with NDCG, MRR. And in order to answer that question, you need a way to quantify your performance. And that's where we're going to see in the next five minutes. What are the metrics that we typically use to do that? So again, this is very similar to if you do search or recommendation. So in case you have a background there, you will see some commonalities.

So here's the setup. You have a bunch of chunks. You do this first and second step. And at the end of the day, you will have k chunks that will come at the top, that you will qualify as being relevant. And you want to compare that with respect to actually relevant chunks. So you can think of it as you have label like, same as in binary classification. So you have relevant and not relevant, and you predict some that are relevant and you want to know well you're doing. So this is the setup.

Well, when it comes to ranking, you need to somehow incorporate this information of how high in the ranking you've put stuff. So here, let's suppose that you have ranked this n chunks from most important to least important. So let's suppose you have first, second, third, and so on, and so forth. And you only care about the first k, because in the rack setting, you typically retrieve the top k that are relevant. And you put all these top k in your prompt. Well, the first metric that you will likely use is called NDCG. It's a lot of letters. I'm going to just explain what that means. So NDCG tries to quantify how good your ranking is by taking into consideration where you ranked relevant documents.

So you have this formula that may seem scary, but it's actually quite simple what it's trying to do. So it's trying to incentivize the score to be higher. If your ranking the relevant documents closer to the first position. So what it does, is it is the sum over the first k positions that you're ranked. And it is looking at whether what you've ranked for each of these positions is relevant or not. So for instance, it checks the first position. First position is irrelevant or not. So relevance-- if it's relevant, it's one. So it's one over some quantity that is a function of the rank. And it does that for all the ranks. So the score will be higher if you rank relevant documents high. So that's basically the goal of this metric.

So this part is the discounted cumulative gain. So it's cumulative gain because you're looking at basically, if you had relevant documents in your first k positions, which is cumulative part, it's discounted because it's better for you to have a relevant document in position one, let's say, than position k. And you can see that in the denominator. Now, why do we say NDCG? What is the normalization part? Well, this metric can take a lot of different values, depending on how many relevant documents there are. So what people do is they compute the quote, unquote, "ideal or optimal or upper bound" DCG that you can get for a given query, and they call that ideal DCG, and they just normalized DCG over IDCG. So the reason why they do that is they want you to score a score of one, if you are matching the optimal ranking. They basically want to make the score meaningful. So does this make sense? Yeah?

So the question is, how do they compute the relevance score? So you can think step one and step two as being a two-step process for you to say which documents you are saying are relevant. So at the end of this two-step stage, the ones that you say are relevant are here. Now, you typically have a score for each retrieved chunk. So you're going to sort these chunks. You're going to sort them by the score in a descending order. And the relevance here that you're going to use is the actual label. So is the chunk actually relevant or not? So you're going to look at your k retrieved chunks, and you're going to ask yourself, OK, is the first chunk actually relevant? So you have a label. You know which ones are relevant, which ones are not. And these ones are going to be the ones you will use in the formula. Yes, that's the ground truth. Exactly. Yeah. Cool. Does that make sense? Yeah. OK. Great.

So you have a bunch of other metrics. You have another one that's called the reciprocal rank. So this one is much simpler. It takes the inverse of the highest rank of all the relevant documents. So if let's suppose in your top k documents, let's suppose the first relevant document, let's say, comes at rank number two. Then rank will be equal to two. So it basically does not care about any relevant documents that are past that first relevant document. So this is just like a simpler metric that typically correlates well. So that's why people use it.

And then, of course, you're familiar with the classic classification metrics-- recall and precision. So if you remember, if you have two classes, you have the positive class, you have the negative class. So recall, what it does is it takes a look at all the actually positive observations, and it tries to ask itself among all these actually positive samples, which are the ones that you actually predicted positive? That's the recall that you all know. And there is, I guess, a ranking equivalent, which is out of all the documents that are relevant, actually relevant, so this is like your positive class, which

are the ones that you actually predicted as being relevant? So this is basically which ones are in the top k?

And similarly, you also have the precision equivalent. So if you remember, precision is out of all the ones that you have predicted to be positives, how many of them are actually positive? So this is the equivalent here. So which are the ones that you've predicted to be positive? So which are the ones that you have selected in your top k? And among those ones, which ones are actually positive? So which ones are actually relevant? Does that make sense?

So I guess these four metrics-- NDCG, MRR, precision at k, recall at k, we see this in a bunch of papers. So I just highly recommend you just get familiar with their ideas and maybe with the formula. And these would be the ones that you would use to quantify, whether your retriever is doing a good job or not. So you have a bunch of benchmarks out there. So there is one that is actually quite popular. It's is called massive text embedding benchmark. So if you want to test your retriever, if it performs well or not would typically take it, and then just evaluate it on that benchmark, and then have all these metrics computed. And then if you have different solutions, you would typically compare this metric.

And with that, we, I guess, hopefully, have a better sense of how to build a RAG system and specifically how to have a good retriever. So we have just maybe one more minute. Is there any questions on that first part? Yeah?

So the question is, so for the re-ranking, we're using an encoder that will output a relevance score. So we will typically train a model that does that. So there are typically-- I believe there are some pre-trained ones, but out there, you can very well have your custom one. So that model will typically be a bit more sophisticated compared to the first step. Because here, you can afford to spend more time to produce that score because you're operating out of, let's say, over 100 possible candidates, as opposed to let's say, much more millions or hundreds of thousands. So that's the idea. Yeah?

So the question is, how about training with contrastive loss? This is a detail, we not I guess, cover here. But in order to train these models, you'll have a bunch of different types of loss function. So I highly recommend you read the S-BERT paper, because in that paper, there are several loss functions that the paper tries to compare. And this is one of them. So that's a great question. I highly recommend reading the S-BERT paper for that. Cool.

With that, I'll give it to Shervine. Thank you, Afshine.

Tool Calling

So thanks a lot for covering the RAG methodology. Tool calling. So now, we arrive at my favorite part of the lecture. We're going to see tool calling and the agentic world. And we're going to see how much more powerful your LLMs are going to become just in a second.

So what Afshine just mentioned is how you would deal with incorporating data that is not structured as part of your prompts to the LLM. And now, we're going to see what we could do more. In the case, the data that we want to inject is structured. So in the case of RAG, you have documents with words and words and words. And you just want to fetch the relevant documents to answer your prompts. But here, let's suppose that you have some structure that determines input/outputs in your data. So typically, you could represent it maybe as a table. So you have separate columns. And then depending on the value of given columns, you have a given output. So we could probably reframe that setup into a function, a setup. So we mentioned tool calling. And then this rephrasing is called the function calling.

So let's suppose that you get this relationship between input and output through a function for the rest of this part. And then here, if I had to transpose what the result of a given ID and field and other arguments would look like, you could interpret it as a function with these as arguments. And the output would be simply what you have as an output to the function. In the world of tool calling and function calling, very oftentimes, you're going to see that LLMs tend to use Python as a language, just because it's so simple to read. So this is what we are going to use as an example as well. But there is nothing that ties us to Python necessarily. So you could well have a tool calling in other languages. Just as a note. Any questions on the setup? OK. Awesome.

So there is nothing controversial about tool calling, but I'm going to still state a full definition to make sure that we are on the same page. So I was browsing here and there, and I tried to find an authoritative source. And this website like on IBM, there is an article that defines-- that tries to define what tool calling is. So I'm going to anchor on their definition from now on. So I'm going to read out loud. So tool calling allows autonomous systems to complete complex tasks by dynamically accessing and may act upon external resources.

So the things that I want you to get from that are two things. So first, you have the notion of completing some task. So given an input and you have to complete some task and then the reliance potentially on external resources. So it doesn't have to be external. You don't have to rely on external resources. But this is one potential property that can help you fill the gap that Afshine was mentioning at the beginning of the lecture, regarding filling the knowledge gap that your pre-trained LLM has. So we're going to see examples in a few minutes, regarding what that could mean. But yeah, this is one magical part of it. OK. Great.

And just to make these statements very grounded in real-life applications, let's just walk through what would a tool call give us in the case of a very specific example? So let's suppose you love teddy bears. You're currently here at Stanford, and you want a teddy bear near you. What if you pull your phone out, and you just ask, find a teddy bear near me. Well, your current LLM, without any tools, would not know some real-life or like real-time update of availability of teddy bears near you. So it would probably answer you something in the flavor of, I don't know or not sure. And I just want to say that with the use of tools, let's see how we could get to a stage where we can inject the information that is necessary for the LLM to know how to respond to your query. And the goal of the next few minutes is going to be for us to figure out, both what we could do what-- what we could inject in the preamble of the LLM? And what would be the steps that we could go through in order to complete such a request?

Does that make sense so far? Yeah? So the question is, are those function APIs pre-computed? So that's a great question. So yes, you define them beforehand. You have some API. And we're going to see that in a second. They're not LLM generated on the fly. OK. Great. Any other questions on the setup? So I know it will be a lot to take in. OK. Great.

So in order to be grounded in real life, let's take a full example of what a function definition could be. So in the case of finding a teddy bear, you could imagine a function definition that is called find teddy bear. And depending on your location, calls some API and retrieves potential candidates. So I'm going to go through the main characteristics of what a function call contains and link it to the definition. So first of all, when we want to display such an API to the model, you need to document it, its input and output, in order for the model to what this function is for. So typically, the description that you have in the example of Python under a given function would be crucial for the model to know what it is about. And then we saw in the definition of tool call that we just gave, that we have the ability to make backend calls. And this is

exactly what we would need to do in the case of finding teddy bears around you. You would need to query some API to retrieve available teddy bears. And based on your location, return the nearest ones.

So this is exactly what this function implementation is doing. It returns something that is well structured. So you see maybe it's small from where you are, but you have some class definition that puts some structure into the output and makes it interpretable. And we're going to see very soon that this output is going to be what the model will anchor on in order to give its final response. OK. Great. And one other thing I will say is all of these that we see here is what you have implemented. You will not see all of these-- if you are an LLM, you will not see all of this. All you care about as an LLM is the function API, the input and output, as well as the main lines of documentations. So all these implementation details, you're going to have it on your code base. But the LLM is not going to see it. OK. Great.

So now, let's go step-by-step into how we could make this work. So the first stage is as you ask a question that is related to your function, you would insert at the beginning of your preamble the function API. And as I mentioned without its implementation. So you just have the function itself, and then a full documentation. So here, you would want the LLM based on your user query to feed the right arguments to your function. s to your point, the goal of the LLM here is not going to be to infer any of the functions implementations, rather only what arguments we should put to it. Yeah?

So the question is, how do you even train this? So we're going to see that in just a few slides. Great point. That's the next question we need to ask ourselves. So we're going to see how we train these. But here, let's say it has been trained. The LLM would have, from its context, probably our localization, because let's say you have activated your location permissions and your LLM knows where you are. So it knows you are at Stanford. And these would be the coordinates being fed to your LLM. And then the second stage is to actually do that function call. So it has nothing to do with LLMs. You just take your function, your argument, and you execute that and you get some answer. And the answer, as we mentioned, is structured in a way that's understandable. So in that function implementation, you return some object that informs characteristics about the return teddy bear. So for example, you have its name, maybe location, and so on.

And you feed that response back to the LLM in order to get a final response. So when you ask to do LLM a given question, you don't want to have this JSON-like response, but rather a response in natural language. And this exactly is the motivation for that

last stage. So I'm going to pause here for a second and check that this three-stage mechanism makes sense to everyone. OK. Great.

And exactly what you were mentioning here is going to be the next focus. How do you even train that? So let me ask you this question. If you were to train an LLM to use this tool, what steps would you need to focus on? So the answer is you're going to feed the API implementation. Yes. So you would need the first LLM call to be somehow recognizing the pattern of the function implementation and the query, and link it to the arguments that you would put in your function. So yeah. Great. So tool prediction.

And do you need a second set of SFT pairs? So you have this last stage that is still LLM-driven, where you have your tool answer, and you need to output a final response. You might say, OK, hey, the LLM has seen a bunch of structured data and knows how to put into words, things that it sees, which is a fair point. But usually, you might want your responses to be formatted a given way. So you might want to also have SFT pairs that do this mapping the way you want it to. So this is why you typically have these two SFT pairs. And if I have to be a bit more precise, the second pair isn't just mapping the JSON response to the final response, it's actually linking all the conversation history so far, so that it knows that the initial query was someone in search of a teddy bear. It knows there has been a tool call, and it knows that the results correspond to that tool call. So it would be a slightly longer input in this SFT pair. Does that make sense? OK. Great. Yeah?

Yeah. So great point. So the question is, what if we have more tools, do we need some tool selection or more examples? So this is a great topic. We're going to see it like in a few slides. Yeah. So it's slightly more complex way of doing things. But if you want a very quick answer, if you go the SFT way, you could show multi-tool inputs. So you could include all of that in your SFT data sets. But we're going to see the topic of tool selection very soon. Great. Any other questions? OK. Awesome.

This is what I just mentioned. Yeah, the conversation history so far is always what you get as part of the input. And then the output is what you would want the LLM to predict at that given stage. And since you're doing SFT here, you don't have just one, but multiple such examples. And you would want your examples to be varied and representing the typical user distribution. So I was asking, find a bear near me. And this example showed to the model how you could ground the information of location based on the user's location, even though you didn't give it. And in this other examples, you could give the model other kinds of instructions, directly saying, I want a bear at that location to teach the model to look at-- like to ground the argument at

different places. And you could expand these sets by also varying the kind of input, so it doesn't have to be worried that way. You could do it multi-turn maybe in the middle of a conversation, you ask for a bear and so on. OK. Great.

But this is not the only way of training a model to do so. So these days, LLMs become more and more powerful in their reasoning. And the kind of data they are trained at pre-training and initial instruction tuning is typically these kinds of code data. So at the end of it, they how to manipulate Python codes very well. So you might ask yourself, is it really needed for me to teach the model how to map a query to a function call? And that is a very interesting observation. And you see these days that you can forego of specific SFT training and try to get around it with only training. So here, instead of writing SFT and then retraining the model, we're going to see that you could actually replace it with only an explanation.

And does anyone have an idea of how we could even come up with such an explanation? So let's say I have a new tool. I have the API. I want my model to use it. How would you go around it? Yeah, that's a great point. So one method could be few shot learning. You just show in the context window samples of input/output. This is a great point. You could definitely do that. And this is typically one accepted practice. But if I told you that few shot learning has challenges when it comes to generalization, because you would need to give specific points as input/output. So it might fit to some cases and not necessarily unnecessarily generalized to the whole span of human language. Is there another way you could go around it? Yeah.

So the answer is ask it to do reasoning. Great point. And writing a prompt that does the reasoning in a way that makes sense is very hard. So in practice, you wouldn't write it yourself. You would take these SFT pairs. So these are the behavior that you want to enforce. And you could use it as some evaluation set. So you could say, OK, hey, if I ask this question, I want that stool call. And you have a set of pairs. And you could run whatever you have so far in terms of explanation. You evaluate these prompts against the evaluation set. So you have some wins and losses. Maybe find a bear in Paris. Doesn't work well. Some other prompts do. So you have a list of each sample with a score. And you could feed it back to a reasoning model, typically, to do the explanation writing for you. So this is a trick to help you avoid doing that hard work.

Basically, showing the model, hey, with the current prompts, here's what we get. What would you have changed in order to make the evaluation results better? So you get with that process an iteration on the detailed explanation. And if you have to see results in practice, you would be surprised at how well it does the explanation for you.

So the takeaway here is I do not recommend you write it end-to-end. Maybe just a draft and you let some very powerful model with excellent knowledge of logic do it for you. Yeah? Yeah.

So the question is, what you just mentioned, is it training or inference? So it's training. So you do it at the very beginning. You want a fixed prompt that explains to the LLM how to use it, how to use that function. So you would typically do that offline. Offline you iterate on the explanation that says exactly how you use the prompt, how you use the function. And then at inference time, you would put that fixed explanation alongside the function API, such that for any query, it knows what to do. Does that make sense? OK. Great. Any other questions here? OK. Amazing.

I just mentioned an example with teddy bears that would be in the category of maybe informational. So you have a question and you want to ask some external API to retrieve that. And you have in reality a lot more use cases that you might see. So one actually mentioned, you have some cutoff dates. And let's say you ask your LLM about news of the day. So you typically wouldn't have anything that comes from the model itself. You would have an API that fetches it from a tool called maybe Search. But you have also other kinds of tools available in the information category. So for example, whether stocks and, let's see, but you also have other categories. For example, if you want to ask the model to do some calculation for you. You could let the model figure it out with some reasoning chain. But one way to get around it is to transform the query into code, execute that code, and then read out the answer. So this is why tool call has a whole span of applications in the computation area. So you have calculations.

And you have another category that we can cite here. So you can take actions on behalf of the user. So let's say you have a tool that sends emails. You could ask the model to send an email for you, and it could put the right things into the header and the message body, and even hit Send for you because you have that component that interacts with the external world as part of the tool API. Yeah. So these are just a few examples, but I just want to say that the field of tool calling is so powerful. You can do anything you want.

And in practice, exactly as you mentioned, you wouldn't have just a single tool API as part of the context. You would have several ones because your LLM is not just an LLM for finding bears. You might want to do other things with your LLM. Maybe you want to hug a teddy bear or check a teddy bear's moods, send teddy bear's a gift and other things. So you have a lot more functions that might be relevant to be added as a preamble. Just because you don't know what function would need to be used, you just

need to put all of that just in case. The setup is exactly the one that you mentioned where you don't have just one API but multiple ones.

Does anyone see any issues with doing so? So the suggestion is that, that's a lot of tools. And then you're right. So typically, if you have too many tools, you have the problem that Afshine mentioned at the beginning of a needle in a haystack, where maybe some tool API will get lost in the context and you don't really what to use. You have a lot of conflicting APIs, maybe, and we're going to see very soon how to overcome that issue. OK. Great.

So I want to take a pause here and summarize how far we've come and some of the drawbacks that we have and see together what we could do to remedy these drawbacks. So first, we saw that we came from an LLM that just responded back at you with a normal words. To an LLM that can actually interact with the outside world, fetch real-time information, or even extends computing capabilities. And exactly like overcomes the issue that Afshine was mentioning the knowledge cutoff one in a different way that than what RAG would do. So you could see these two methods as being complementary. And yeah, both try to do the same thing in some sense. But exactly as someone mentioned here, if you have more tools, then your context window might have things that it doesn't need. And then if you try to support so many cases, you might end up being mediocre at all of them. So this is an issue. And even if that wasn't an issue, you still have the context window that is finite. So let's say you have hundreds of millions of users that use your LLM and that want to use to do a lot of things. You cannot support everyone's use cases at once, just because you couldn't possibly fit all of such tools in your context. And then last, when I mentioned these tools, these are things I hand wrote. And maybe each LLM has their own way of defining and using tools. We're going to see if there is a way to standardize, maybe the way we define and use tools later on. So do these drawbacks make sense. Yeah. OK. Great.

Tool Selection

Tool selection. So let's move on to tool selection. So let's see what we could do in order to make a tool use more scalable. So I'm going to cite here a technical paper from Google DeepMind, which uses a tool selector system. So it functions in two steps. So first you have your query and you have a list of tools that can be as big as you want. And a list of tools only contains the API name and, maybe one or two words about what it does. So you have this prompt and the list of tools. And we ask the LLM to pick the tools that might be relevant. So this is why the technical paper calls this

system tool selector. You could even see the term router being pronounced in the literature. So tool selection, routing are similar concepts. And the goal here is to restrict the number of tools to only those that might be useful. And in the second stage, you take all the selected tool APIs and feed only those to the context alongside your query. And this is one way you could take to overcome this issue of having too many tools at once.

Is everyone convinced that tool selection could be a good way to fix the problem here? So the question is that basically RAG? You could do it with RAG. That's a great point. But it doesn't necessarily have to. So you could have an LLM that does this job, just picking the right tools. And you ask some instructions to the LLM to output the right answer. You could definitely do it with RAG. That is just a great point. Yeah. So does it make sense to everyone? OK. Great.

And I see time is flying. So we're going to go very quickly on the standardization issue. So as I mentioned, every tools implementation, like the way you specify a tool implementation can be bespoke to a given LLM. And you wouldn't want that. Because for every LLM, you might need to implement always these tools over and over again. And duplication is not what you want.

Model Context Protocol (MCP)

This is why there is a standard called MCP, Model Context Protocol (MCP) that I know a lot of people talk about it. So it's something from the Anthropic team that standardizes the way tools are exposed to models. So it's a protocol. It's called a model context protocol. And it defines a standard way of presenting these tools. So I'm going just to say the very big lines of it, so you have this kind of vocabulary as part of MCP. So you have an MCP server, which is the instance that serves tools. And then tools are implementations of the functions that you want people to use. Prompts are templates that could show the user how to use these tools. And then all of these can anchor on resources, which are external databases that you can use to complete the task. And then an MCP server, when you use it, has a one-to-one connection with a piece of infrastructure in your LLM host that is called MCP clients. But these are maybe infrastructure details.

And I just want to ground what I just mentioned in reality. So we know our teddy bear loves to read poetry. So in the case of recommending a poetry book to our teddy bear, you could think of a MCP server that provides tools that are linked to books. So your LLM hosts here-- so let's just take the example of Claude since MCP is from

Anthropic. So Claude would be your LLM host. Your MCP server would be probably implemented by your book provider. Because typically, they are the most experts as at serving such content. And we could assume that the book provider, MCP server has tools regarding finding books or recommending them. And then the prompts here could be a ways to show you how to find a given title, or maybe recommend it with respect to some flavor, maybe with respect to the user's test taste. And some examples of resources here could be the teddy bear's personal collection, or maybe top books that people buy, just as an example. OK. Awesome.

Agents with ReAct

So now, I'm going to come to potentially the most exciting part of today's lecture, Agents with ReAct which is agents. So we saw how much more powerful LLMs could become with tools. Agents could be seen as one layer up from it. So I'm going to give first a definition, just so that we agree of what agent means here. So it's a system that autonomously pursues goal and completes tasks on a user's behalf. So compared to tools, you not only can perform tasks, but you have also some reasoning involved in it. So you could have multiple loops of iterations. And this is what differentiates an agent from a tool in plain terms. When usually people talk about agents, it has that recurrence or higher level of reasoning baked into it. And so I'm going to contrast like the agentic world with respect to the world we have seen, where you may have multiple calls to tools. And these new agentic framework is not necessarily disjoint from the ones that we presented before. You could well have reasoning chains inside of it. So it's like a potentially overlapping. But the structure consists of tool calls and iterations. OK. Great.

And now, I'm going to talk about a hallmark paper called ReAct, to reason plus acts, which decomposes possible loops into different stages. So very oftentimes, when you have a query and you want to pursue a goal, you cannot do it one shot. You need somehow to decompose the goal into actionable substeps, perform each of these steps, and then come up with the answer. And this is exactly what ReAct is about. It's about decomposing complex tasks into loops of things that can be done atomically. So here, I just want to say one thing. We decompose these steps into observe, plan, and act. But it doesn't necessarily have to be called that way or be in that order. For example, the ReAct paper, I think, introduces terms think, observe, act. You might see the language changing a bit from paper to paper to paper. But the high-level intuition remains.

So let's just ground on a very specific example. So to be in the theme of these days weather, it starts becoming a bit cold. And our teddy bear might be cold in its home. So let's just have that as an input. My teddy bear is cold. Please, do something. And let's see what an agentic workflow could do about it. So first, you have this observe stage, which translates the user query into an actionable formulation. So when you say that your teddy bear is cold and you ask it to do something like here, in this observe stage, you would link it back to the notion of temperature. The user's teddy bear is cold, which may be due to the current temperature of the room, which is currently unknown. So right off the bat, you now know that you need to do something with temperatures.

So this brings us to the next step called Plan, where that something is unknown, and you need to find it. Plan will spell that out for you. So here, you want to determine the temperature of the room. And luckily, among your tools, you might have something that does something in these lines. And this is where you use it. So the Act stage is using all these APIs that might be in the context. So for example here, if you have get current room temperature, this is the way to use it. And as I mentioned, these tools are useful to the LLM when you output information back to it that it can interpret. So this tool returns the temperature. And you need now to interpret what the temperature means.

So this is why you go back to the observed stage. You describe what the world looks like and what you should do about it. So here, you can read that the temperature of the room is cold. It's 65 Fahrenheit, and the observed stage might indicate that it's colder than expected. So this is why you need to plan something. And then at the Plan stage, you're like this, this was colder than expected. So I need to increase the temperature. And then you go back to the Act stage. Luckily, how to increase the temperature. And it has a function that can have the temperature increase arguments and put it to it, so you can adjust the temperature. And then once it's done, the observed stage concludes that the temperature is not set to the correct temperature. And this is the time where you can exit the loop and return the response to the user.

So here, we have increased the temperature by 5 degrees. And then the output reads that back to the user in the hope that it has fulfilled the user's query. So what I described within the loop is what makes a workflow an agentic one. So you have some initial query. You have some actions that you can perform. And then at each stage, the LLM tries to see if it has reached the goal yet. And then if it has, then it will go to the output stage. Otherwise, it may have more reasoning loops, where it does more work. Does the definition of agent that I propose here make sense? OK. Great.

So now taking this a bit further, you could think of way more than one agent. You could have an agent for setting the thermostat. But maybe in your home, you want to manage how energy is distributed, or air quality, if you have some settings to tune. So you can have different agents. And one interesting use case is to have the user say something to an agent, and potentially have all of them communicate together. So these prompts potential needs for also a standardization of communications between agents, similarly to what we have seen between LLM hosts and tools. And this is what prompted Google to release the Agent2Agent protocol earlier this year.

So yeah, highly recommend taking a look at their document specification. But just to give the main lines around it, you have some standardization of what an agent can expose. So it typically exposes a set of skills. So it can do things. And then it gives examples about it so that other agents are aware of it. And what you need to do as a developer would be to define these skills. And also, another key thing is to define how the agent executes a given request. So for example, when you execute a given query, what is the status that you emit to other agents? Or there is a cancel method, as well, where let's say an agent says, stop what you're doing. What is the process to cancel an action? So these are some of the main functions like the Agent2Agent protocol asks people to fill out. Yeah?

So the question is, is each agent an LLM with some context to do some tasks? So you could definitely imagine it be the case. Typically, in the example that I mentioned here, yes. And the agents operate independently. So they have their own reasoning loops. And all the other agency is the input and output. Yeah. So the remark here is maybe your token budget can go all over the place. So you can have some budget restrictions put in place. This is a good point. But each of these agents, they would not eat on each other's budgets. It might eat on your money budget, but this is indeed a concern.

Safety and Closing Thoughts

So I'm going to move on to the topic of safety, Safety and closing thoughts which I have not mentioned so far. But that is very important because with these new capabilities comes a whole string of new potential issues. And you have seen these models. They have now the ability to execute actions for you. So you could think of some harmful actor doing things that you wouldn't want for you. And I give one example here that could be a concern-- data exfiltration. So let's say you have access to a tool that can write in a public visible way data. So let's say you have, I don't know some email agent. So if you have a prompt that, for example, says, write my password,

which potentially the tool could have access to to an email to that address, you could exfiltrate data that belongs to the user out of it. So this is typically one risk that you might have. You can have other safety risks. And I link the paper here that goes through some of them. So tool sort. So I recommend a read.

Now, you might ask, what could you do to get around it? So you have typically two classes of remediations. One that might come at the training stage where, if you recall the training process of R1 or even other models, you have these harmlessness components. So you typically have data as part of the data mixtures that you train during SFT and reinforcement learning that could cover safety. So this is where you could remedy these issues. You have another option. Let's say, you have a query that has gone through your lines of defenses, your training lines of defenses. You can also have inference safeguards. For example, safety classifier that looks at the conversation so far, and that judges whether the output of the LLM is going to be safe or not. So you have even that as a safeguard.

And just as a pointer, I'm going to talk about that agent safety bench, which summarizes the span of possible safety hazards and offers a benchmark, a full suite for it. So people can refer to these benchmarks to know whether their LLM is safe. And it's a very important topic. And just yesterday, Anthropic revealed that there were the victims of a large scale cyber attack launched from Claude. So this is an issue that is-- so it was also using tools and agentic capabilities, and they published super detailed report, saying exactly what the attackers did and went step-by-step through possible lines of remediations. And this is just to emphasize how important safety is in this more capable world. And both attackers and the line of defense can be more and more sophisticated. So it's not a lost battle, it's just that the tools we have for defending against like tool-based attacks need to be-- so we need to have such measures in place. And with these measures, maybe it's going to be fine. So I think that was the overall mindset of that article, which I strongly recommend to read.

I'm going to say a last few words. So when you talk about agents, you have this risk that at every step of your thought process, you might diverge to something that just doesn't work. So that is a huge problem. Let's say if the model doesn't grounds to the output properly, or maybe does a mistake into the argument prediction of a tool call, so this is one big issue, which is the reason why you don't see large scale agents ruling the world right now, because it's really like, we are limited by these consequences. And then I'm going to talk about the developing capabilities of models that enable you to have this agentic capabilities. So this is something that you can fix with SFT. But ideally, you wouldn't want to use SFT to fix reasoning gaps and use the model itself.

And we're going to see next week what the evaluation landscape looks like. So I'm going to reserve it to next week.

And just a few words of advice regarding building tools or building agents, always start small on a very simple case. For example, find the nearest bear. Try to see if your current implementation and prompts just works and then start from there. So start small and then start smart. So take the most capable model first so that you know the headroom of where you can go with the current models, and then try to gain on latency capabilities and so on. Start correct, start small, and then you can optimize later. And when it comes to debuggability, these LLMs they output chains of reasoning. So it's always good to look at them, to see what is going wrong.

And I will close this lecture on a note. My favorite use case of using agents right now is as an AI assistant coding. So this is something I strongly recommend in case you have a project that requires you to do complex piping of things. You can free your mental load by delegating some of these tasks. And this is something that is you're going to see in the real world, very much used now, but with the caveat that, please make sure to learn the foundations of codes, know how to code right, because your taste is going to matter most from now on. Generating code is cheap, but judging whether a code is correct, and does the right thing, this is the hard part.

And with that, thank you.